

Assignment 01: Q1(c) and Q1(d)

Mahnum Ahmed

Areeba Izhar

Fall 2026

1 Q1(c): Comparison of A* and Dijkstra

Both algorithms were tested on the same tests for the robot navigation problem and courier delivery problem. The result of each test was the cost of the route, the number of nodes expanded and the route itself. “Saved” is how much less work A* did than Dijkstra:

$$\text{Saved (\%)} = \frac{\text{nodes Dijkstra expanded} - \text{nodes A* expanded}}{\text{nodes Dijkstra expanded}} \times 100$$

1.1 Robot Navigation

The tests were conducted on three different types of grid: The 5×7 example grid from the assignment, an empty 10×10 grid and a grid of size 7×10 with a long wall in the middle of the grid. Routes are represented in terms of moves (U = up, D = down, L = left, R = right).

Grid	Start → Goal	Cost	A*	Dijkstra	Saved (%)	Route (A*)
example	(0,0) → (4,6)	10	12	24	50	RRDDRRDRRD
example	(0,0) → (4,0)	8	9	13	31	RRDDLDD
example	(0,0) → (2,4)	6	7	9	22	RRDDRR
example	(4,0) → (0,6)	10	22	25	12	UURRRRUURR
open	(0,0) → (9,9)	18	100	100	0	DDDDDDDDRRRRRRRRR
open	(0,0) → (0,5)	5	6	21	71	RRRRR
open	(2,3) → (8,7)	10	35	92	62	DDDDDDRRRR
wall	(3,1) → (3,7)	12	42	48	12.5	RRRDDRRUUUR
wall	(0,0) → (0,9)	21	64	64	0	RRRRDDDDRRUUUUUURR
wall	(2,2) → (2,4)	2	3	13	77	RR

Table 1: Robot navigation results. A* and Dijkstra columns show nodes expanded.

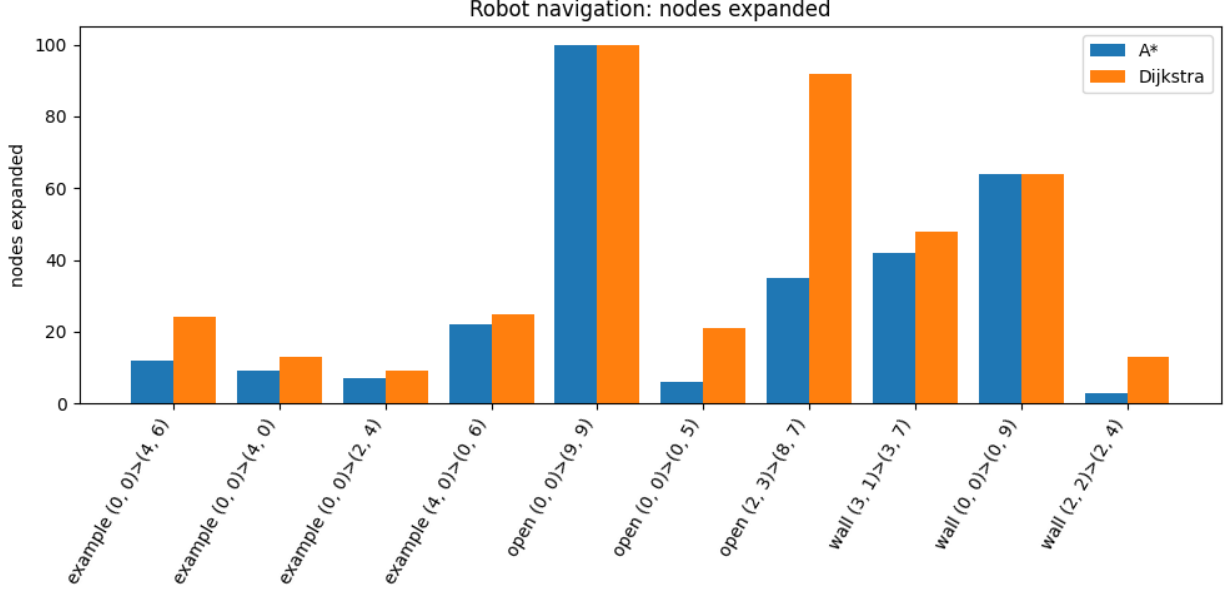


Figure 1: Nodes expanded by A* and Dijkstra on the robot navigation tests.

1.2 Courier Delivery

The tests used trips from the hub (Saddar) to several areas, and a few trips between other areas. Costs use our road type multipliers: main road $\times 1$, standard road $\times 1.25$, narrow lane $\times 3$.

Start	Goal	Cost	A*	Dijkstra	Saved (%)	Route (A*)
Saddar (Hub)	Clifton	8.0	2	4	50	Saddar $\rightarrow$ Clifton
Saddar (Hub)	DHA	14.0	4	9	56	Saddar $\rightarrow$ Clifton $\rightarrow$ DHA
Saddar (Hub)	PECHS	7.5	3	3	0	Saddar $\rightarrow$ PECHS
Saddar (Hub)	Gulshan-e-Iqbal	13.5	4	7	43	Saddar $\rightarrow$ PECHS $\rightarrow$ Gulshan-e-Iqbal
Saddar (Hub)	Nazimabad	12.5	5	6	17	Saddar $\rightarrow$ Liaquatabad $\rightarrow$ Nazimabad
Saddar (Hub)	Baldia Town	28.125	12	17	29	Saddar $\rightarrow$ Liaquatabad $\rightarrow$ Nazimabad $\rightarrow$ SITE Area $\rightarrow$ Baldia Town
Saddar (Hub)	Lyari	15.0	6	10	40	Saddar $\rightarrow$ Lyari
Saddar (Hub)	Korangi	26.5	12	15	20	Saddar $\rightarrow$ Clifton $\rightarrow$ DHA $\rightarrow$ Korangi
Clifton	Korangi	18.5	5	7	29	Clifton $\rightarrow$ DHA $\rightarrow$ Korangi
Malir	Baldia Town	35.875	14	16	12.5	Malir $\rightarrow$ Gulshan-e-Iqbal $\rightarrow$ PECHS $\rightarrow$ Nazimabad $\rightarrow$ SITE Area $\rightarrow$ Baldia Town
Orangi Town	DHA	40.375	13	15	13	Orangi Town $\rightarrow$ SITE Area $\rightarrow$ Nazimabad $\rightarrow$ PECHS $\rightarrow$ Gulshan-e-Iqbal $\rightarrow$ DHA

Table 2: Courier delivery results. A* and Dijkstra columns show nodes expanded.

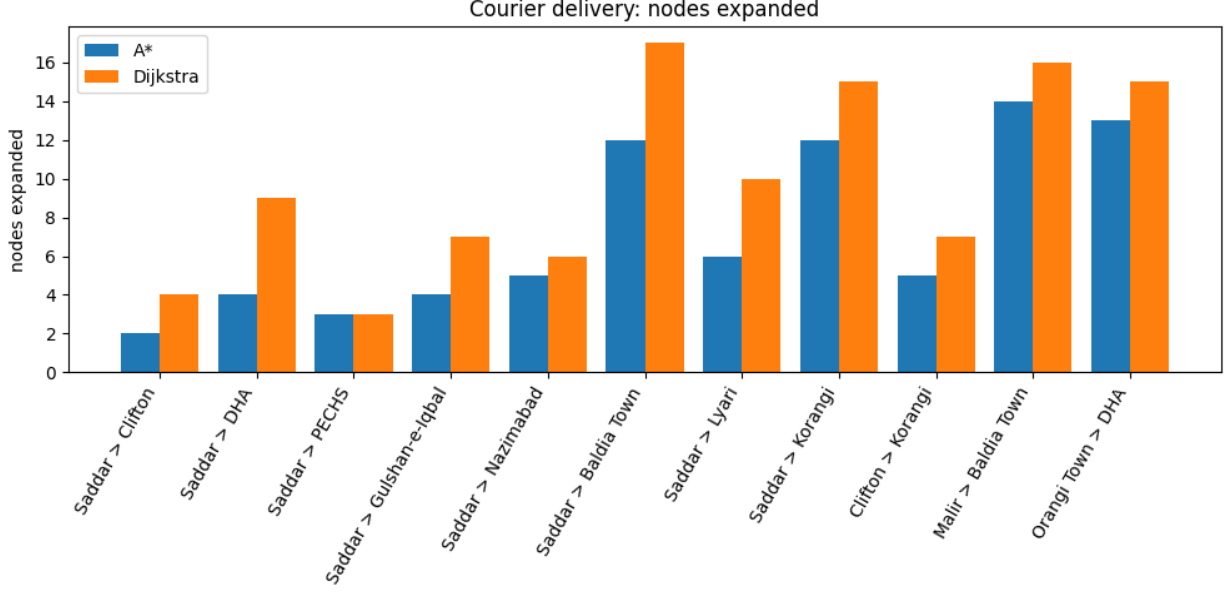


Figure 2: Nodes expanded by A* and Dijkstra on the courier delivery tests.

1.3 Effect of Road Types

We ran A* from the hub twice for the same destinations: once with every road costing just its distance in km, and once with our road type costs.

Goal	Setting	Cost	Route
Baldia Town	plain km	14.0	Saddar → Lyari → Baldia Town (14 km, 2 narrow lanes)
Baldia Town	road types	28.125	Saddar → Liaquatabad → Nazimabad → SITE Area → Baldia Town (22.5 km, 0 narrow lanes)
Orangi Town	plain km	20.0	Saddar → Lyari → Baldia Town → Orangi Town (20 km, 3 narrow lanes)
Orangi Town	road types	26.875	Saddar → Liaquatabad → Nazimabad → SITE Area → Orangi Town (21.5 km, 0 narrow lanes)
Korangi	plain km	24.0	Saddar → Clifton → DHA → Korangi
Korangi	road types	26.5	Saddar → Clifton → DHA → Korangi

Table 3: Same trips with and without road type costs.

1.4 Observations

- **Both algorithms always found the same cost.** This holds in all 21 tests, because our heuristic never overestimates (Manhattan distance for the robot, straight line distance for the courier).
- **A* expands fewer nodes when the goal is straight ahead.** For instance, open (0,0) to (0,5) with A* expanded 6 nodes while Dijkstra expanded 21 nodes; open Saddar to DHA expanded 4 nodes and Dijkstra expanded 9 nodes.

- **A* helps little when walls block the way.** In the wall grid, (0,0) to (0,9) is only 9 steps away by Manhattan distance, but costs 21 to traverse due to the wall; the heuristic is too optimistic and both algorithms expanded 64 nodes.
- **A* can have nothing to skip.** Both algorithms expanded all 100 cells in the open grid (0,0) to (9,9), where every cell is on some shortest path.
- **Courier trips to far areas save less.** the road type multipliers make the real distance much farther than the straight line distance (e.g. Orangi Town to DHA: 13 vs 15).
- **Road types change the route.** To Baldia Town, plain distance goes through the narrow lanes at Lyari, but with road type costs the courier takes a longer route with no narrow lanes at all.
- **Effect of the heuristic.** It never changes the answer, only how many nodes are expanded. It helps most when the straight line distance is close to the real cost.

2 Q1(d): Route with Stopovers

The van starts at the hub, must visit every stopover in some order, and then returns to the hub. There are only a few stopovers, so we find the cheapest order by trying all of them. The cost of each leg (going from one location to the next) comes from A*, and we run each pair only once and reuse the answer.

Algorithm 1 Route with stopovers

Require: hub, list of stopovers, road data

Ensure: full route, total cost, order of stopovers

```
1:  $points \leftarrow [hub] + stopovers$ 
2: for all pairs of locations  $(a, b)$  in  $points$  do
3:   run A* from  $a$  to  $b$ 
4:    $legCost[a][b] \leftarrow$  cost of the route found
5:    $legRoute[a][b] \leftarrow$  route found
6: end for
7:  $bestCost \leftarrow \infty$ 
8:  $bestOrder \leftarrow$  none
9: for all orderings of the stopovers do
10:   $total \leftarrow legCost[hub][first]$ 
11:  for all consecutive stopovers  $(a, b)$  in this ordering do
12:     $total \leftarrow total + legCost[a][b]$ 
13:  end for
14:   $total \leftarrow total + legCost[last][hub]$ 
15:  if  $total < bestCost$  then
16:     $bestCost \leftarrow total$ 
17:     $bestOrder \leftarrow$  this ordering
18:  end if
19: end for
20:  $route \leftarrow [hub]$ 
21:  $stops \leftarrow [hub] + bestOrder + [hub]$ 
22: for all consecutive locations  $(a, b)$  in  $stops$  do
23:   add  $legRoute[a][b]$  to the end of  $route$  (without repeating  $a$ )
24: end for
25: return  $route, bestCost, bestOrder$ 
```

important thing we noticed: -> If there are k stopovers, there are $k!$ orderings to try. This is fine for the small number of stopovers in a delivery run.

-> A* is used for every leg, so each leg is the cheapest route between its two locations. Trying all orderings then gives the cheapest full trip that visits every stopover and returns to the hub.

-> A route between two stopovers may pass through other areas on the way. That is fine, the stopover is still visited in its place in the order.